

00:00 next we have is routing in the previous
00:02 video we talked about different HTTP
00:04 methods and how they are important in
00:07 the HTTP semantic so in a way all these
00:10 HTTP methods they describe your intent
00:13 which we already talked about and you
00:15 can say these HTTP methods they express
00:18 the what of a
00:21 request your intent your action what do
00:24 you want to do on that particular
00:26 resource or what do you want to tell
00:28 your server that your intention is
00:30 whether you want to fetch some data or
00:32 add some data or update some data or
00:34 delete some data and the role of routing
00:36 is it expresses the where of a request
00:41 where do you want to send your intention
00:44 where or which resource you want to
00:46 perform your action or your intent on
00:49 you need to tell the server where do you
00:51 want to go for example you have a
00:53 request and the method is get right your
00:57 intention is fetching or getting some
00:59 data from the server
01:01 and your route path or the URL path is
01:05 slash
01:08 users and the server sends you an array
01:11 of
01:13 users okay so what you're saying is I
01:16 want to fetch some data you want to
01:19 fetch some data your action is fetching
01:21 and from where or what kind of data
01:24 right this is the resource this is the
01:27 route whatever you want to call it and
01:31 the combination of these two your
01:34 intention your action and your route the
01:38 URL parameters the address the server
01:41 takes this and this and Maps it to a
01:44 particular Handler or a set of
01:46 instructions and it performs all the

01:48 business Logic the database operations
01:50 and whatever is needed and returns you
01:53 the data and to summarize it you can say
01:55 that routing is basically mapping URL
01:58 parameters to a server side logic and
01:60 and that's all there is to it and here
02:02 we have our familiar BB Suite interface
02:05 and we have a react app which we will
02:08 use to demonstrate some of these routing
02:11 Concepts so I'll just go ahead and fire
02:13 some apis and then we can jump in and do
02:15 some analysis so the first request was a
02:18 get request and if you look at the
02:21 request and response this part we are
02:23 already familiar with this is the HTTP
02:26 method this is our intent or the action
02:28 we want to perform which is get or
02:31 patching now the this part this is what
02:36 we call the route right the address of
02:39 our request where do you want to go in
02:41 the server which is in this case / API
02:44 SL books books is called a resource and
02:48 we'll learn more about resource when we
02:50 study about rest API concepts for now we
02:52 can just assume this is a route right
02:54 and this is a route so this is our
02:56 action this is the address you want to
02:58 go to in the server and when we send
03:00 this request and the request goes
03:02 through the server responds with this
03:04 data right whatever the responses it
03:07 does not matter and for the sake of this
03:09 demo we are just sending some fillers
03:11 but typically this is what happens you
03:13 send a request the server does some kind
03:15 of authentication if it's there then it
03:17 sends the data and then in the next
03:19 request which is a post request here the
03:21 intention is post the HTTP method is
03:24 post the route is still the same/ API SL

03:28 books the request goes through it
03:30 creates another book or whatever the
03:33 resources and it returns all the
03:35 resources again in the response right so
03:38 if you notice here in the previous
03:40 request and in this request this route
03:43 part is the same so what happened is in
03:45 the previous request the request was a
03:47 get method and the route was SL API SL
03:52 books and in this request it's a post
03:55 method right and the route is still the
03:58 same/ API Das books so what happens in
04:01 the server is these two things the
04:04 method and the route these two things
04:07 are kind of a key which map to a
04:10 particular Handler in the server right
04:12 the server first checks what the method
04:14 is then it checks the route and it
04:17 concatenates these two and forms a
04:19 unique routing logic right this is a
04:21 unique path and this is another unique
04:24 path these two will never Clash right
04:26 these methods they differentiate between
04:28 the two routes and now that we know what
04:30 what basic routing is we can jump into
04:33 different types of routing right and the
04:35 first two examples that we discussed
04:38 the/ API SL books for get and post these
04:41 kind of routing or routes are called
04:44 Static routes right and why are they
04:46 call static routes for obvious reasons
04:48 because they don't have any variable
04:50 parameters inside the route and what do
04:52 I mean by variable for example this part
04:54 SL aa/ books this will stay consistent
04:57 this is a constant right we don't have
04:59 to think about a dynamic parameter
05:02 inside this route this is constant we
05:05 are always going to use this string /
05:07 API / books whenever we make the request

05:10 nothing changes in that and it is always
05:12 going to return this kind of response
05:15 that's why it's called a static route
05:16 it's a constant or it's a particular
05:19 string which never changes and it always
05:21 returns the same kind of data so in that
05:23 case what is a dynamic route so let's
05:26 fire these two requests now
05:31 and if we look at the first one so here
05:33 if you see the method is get and the
05:35 route is / API SL users sl12 3 and in
05:39 this case this is supposed to express
05:41 the ID of the user the application of
05:43 this API endpoint is we can fetch
05:45 details of a one particular user using
05:48 the ID of the user inside the route
05:50 parameter and when we make this request
05:53 the server can extract this particular
05:55 ID from the route path and it can do
05:58 whatever operation it wants to do like
06:00 fetching the user from the database and
06:02 returning the details of the user in
06:04 this case we just the server is just
06:06 fetching the user ID and returning it in
06:08 the response to express that it got the
06:10 user ID from the route since in this
06:12 video we are not going to get too much
06:14 into the code and we just want to
06:16 understand the concepts behind it but
06:18 just to give you an idea this is what
06:20 the the route matching part in the
06:22 server looks like the server is
06:23 basically saying let's say the Handler
06:25 is called route or let's say it's
06:28 called R right R do
06:33 get and in the matching part it is
06:36 saying it's a string slash API SL
06:42 users slash colon
06:49 ID this part matches the method in the
06:52 server and this part this part matches

06:55 the route so the server is saying if any
06:58 request with the method get
07:00 and with the route slash API SL users
07:04 and in the next part any kind of string
07:08 right this Fallen ID it basically means
07:10 any kind of string that comes in this
07:13 format routed to this Handler whatever
07:15 Handler is waiting to respond to that
07:17 kind of route right so the thing to
07:19 remember is uh this convention you'll
07:21 find in any kind of servers it does not
07:23 matter it's Java or python or nodejs or
07:26 goang rust it does not matter this just
07:30 a convention find this is what I've seen
07:32 the industrywide practices mostly colon
07:34 which says it's a dynamic parameter that
07:37 the user is going to send in place of ID
07:39 the use of this is when the request
07:40 comes in our case request was / API SL
07:44 users and /1 23 the request got routed
07:48 to this part it matched with the get
07:51 right we we were sending a get request
07:54 and in the next part it matched the
07:55 first / API then the next one/ users and
07:60 it's a string right it looks like a
08:01 number but in route parameters or in
08:05 route paths whatever uh number special
08:08 characters everything is converted into
08:09 a string so this thing this gets
08:12 inserted into this slot this Dynamic
08:15 parameter slot which we are calling as
08:17 ID if you read it it's clearly readable
08:20 right you can read it like you're saying
08:22 patch me some data because it's get and
08:26 where is that data that is in/ API SL
08:29 user
08:30 and whose data that is it is with of the
08:33 user with ID 1 2 3 right it's clearly
08:36 readable that's the whole idea of rest
08:38 apis because it it provides a human

08:41 readable construct to routing this is
08:44 called a dynamic route and the
08:46 terminology that we use is this part is
08:48 called a route parameter or path
08:50 parameter that is something to remember
08:52 we have like two different types of
08:53 parameters that's a little confusing one
08:56 is these kinds of parameters these kinds
08:58 of dynamic parameters are called path
09:01 parameters because they go right after
09:02 the forward slash right it is part of
09:05 the route that's why they are called
09:07 path parameters or route parameters and
09:09 the next one we have query param so for
09:11 this we can just type out something
09:13 let's
09:14 say some value and let's hit this
09:20 API and we if we look into this API this
09:25 is what it looks like this is the method
09:27 and this is the route right this whole
09:30 part is the route basically the route
09:32 is / API SL
09:37 search question mark
09:41 query equals
09:43 to sum plus
09:47 value and now I realize this is more
09:49 readable here than here but anyway we'll
09:53 make do so this part we understand right
09:55 this is the route / API SL search this
09:57 is the address that we want to go into
09:59 the
09:60 this is the part that server uses to
10:04 match including the method and this
10:07 route this is the part the server uses
10:09 to match it to some Handler and it
10:11 performs the logic and returns some data
10:14 which is not important the thing to
10:16 focus here is the query part here the
10:18 question mark and the key and the value
10:22 this is called query parameter that's

10:23 the terminology and next is the
10:25 application why do we use it because in
10:29 post requests or in put requests we have
10:32 the body of the request right we get to
10:35 use the body and we can use the body to
10:38 send some kind of data send parameters
10:40 that we want to attach into the request
10:42 we have some kind of container to send
10:45 some user defined values into the server
10:48 but in case of get request in rest API
10:51 get requests don't have a body you want
10:53 to send some value in the get request so
10:56 let's say you can say that we can send
10:59 those values in the path parameters
11:01 right but path parameters serve a
11:04 particular purpose right they serve as a
11:07 semantic expression right in the
11:09 previous request that we saw we were
11:11 saying SL API SL users SL1 23 and the
11:16 semantic meaning here is we want to
11:18 fetch the details of a user whose ID is
11:21 1 2 that is the semantic expression in
11:23 the path parameter but if we want to say
11:25 we want to call this API which is / API /
11:28 search and we want to say the search
11:31 value is some random value uh that the
11:33 user has typed into the input box how do
11:35 you want to send it let's suppose we
11:37 want to send it in the path parameter
11:39 what will it look like SL API SL search
11:42 and Slash the value in this case it will
11:45 be some
11:47 value some value technically technically
11:52 it's possible you can do this but it's
11:55 it's very hard to maintain and it it it
11:57 defeats the whole purpose of rest API
11:59 right providing semantic Expressions to
12:01 AP end points it defeats the whole
12:03 purpose that's why we have this concept
12:06 called query parameters in query

12:07 parameters we can send a set of key
12:10 value pairs in a request typically we
12:13 use it with get request because we don't
12:15 have a body to send data in so in get
12:17 request we get these thing called query
12:20 parameters which are key value pairs
12:23 that we can send to the server to let to
12:26 send some kind of metadata about the
12:29 request for example let's say it's a
12:32 request about fetching paginated data
12:34 you want to petch a list of books which
12:37 are paginated we have not gotten into
12:40 pagination yet but I'm pretty sure you
12:42 must have encountered it at some point
12:44 they basically look like this so let's
12:47 say it's a endpoint which fetches a list
12:50 of books in paged format so it looks
12:54 something like this SL API SL books and
12:58 when you hit it the server returns an
13:01 object or a Json and inside data you
13:06 have whatever the a list of an array of
13:10 books and it returns some a set of
13:14 metadata about the response right and it
13:17 looks something like this like
13:22 total current
13:24 page and total
13:27 Pages things like this it depends on the
13:29 implementation but something like this
13:31 so the purpose of this is and you also
13:35 have a limit here purpose of this is the
13:38 server will paginate the data the
13:41 response and it will send a chunk of
13:44 data let's say the limit is 20 there's
13:46 some kind of default limit so what the
13:48 server will do is it will send 20 books
13:51 from the start and it will let you know
13:54 how many books are there in total let's
13:56 say 100 books and what is the current
13:58 page which is the page one and what are
13:60 the total pages and if you divide 100 by

14:02 20 it will be like five pages right
14:04 these are all the kinds of values you'll
14:06 receive from the server and the client
14:08 can use these parameters use this
14:10 response metadata to make the subsequent
14:13 requests accordingly according to the
14:16 client's needs how how can it fetch the
14:18 next page in the next page it can do/
14:21 API SL books and in the query parameter
14:25 it can do page equals to two right
14:28 because by default the server send the
14:30 page one we did not have to send any
14:32 request any query parameter for the
14:34 value of page or limit but in the second
14:36 request we want do we want to tell the
14:38 server that we want the response for
14:40 page two so in a get request we can send
14:43 that information using query parameters
14:45 this is one application usually API send
14:46 different kinds of parameters in the
14:48 query for example you want to filter by
14:50 some user defin parameter you want to
14:53 sort it what is the order you want to
14:55 sort in whether it is ascending or
14:56 descending all these kinds of
14:58 information will go inside query
14:59 parameters in in the form of key value
15:02 so that is basically the uses and the
15:05 definition of quy parameters and this is
15:08 what the API typically looks like next
15:10 we have is nested route this is not
15:12 really a type of routing it's just a
15:16 practice that you will see everywhere
15:18 because in rest apis for a semantic
15:20 expression we often have to resort to
15:23 nesting right nesting different types of
15:25 resources and the nested route is
15:27 typically the result of that and if we
15:29 hit this API and want to see what it
15:32 looks like this is what the request and

15:33 response looks like so what it says is
15:37 is is the get request and this is the
15:39 route uh we can say this is the static
15:41 part of the route and this is one
15:43 Dynamic route parameter or path
15:45 parameter and this is the second Dynamic
15:47 path parameter and the application of
15:49 this is and why do we do this because
15:53 again to express the semantic meaning so
15:55 what we are saying here is we want to do
15:58 a get operation
15:60 on
16:01 this route right let's say/ API /
16:06 users with the user whose ID is 1 2 3
16:10 you want to fetch the details of this
16:13 user so semantically the first part
16:15 expresses that we are fetching
16:17 information or data which is related to
16:20 this user and the second part says we
16:22 are again fetching the posts of that
16:24 user and to go one level deep again you
16:28 are fetching a a particular post which
16:30 is the post with ID 456 we can stop at
16:34 different phases here okay let's see so
16:36 the first part the static part is/ API
16:38 SL users this is a static part and this
16:42 in itself is one route which we saw
16:46 earlier it it it returns a result right
16:49 if you do/ API SL users it will return
16:52 list of all users because that's the
16:54 Handler of that match is doing right if
16:57 we go one level deep again/ API SL users
17:02 SL1 23 and this is a unique route in
17:06 itself because we added a dynamic
17:08 parameter and in the server s side it
17:10 will see the get method and it will
17:12 match with a different Handler right and
17:14 that Handler will return the information
17:17 of this user whose ID is 1 12 3 so this
17:22 is one level of nesting we can go

17:25 further which is/ API SL users SL SL1 23
17:30 and/
17:33 posts and now at this point what the Ser
17:35 will see it's a get method and we are
17:39 it will match with a Handler which
17:42 returns all posts of a particular user
17:46 because we stopped here right and this
17:48 expresses a different meaning
17:51 altogether we are fetching the posts of
17:55 a user with ID 1 2 3 all the users posts
17:58 right and in this demo what we are
18:01 doing is SL API SL users SL1 23 SL posts
18:10 SL
18:11 456 and what this is saying we want to
18:16 fetch data of user with ID 1
18:19 123 what do we want to fetch we want to
18:22 fetch the posts and not all post we want
18:25 to fetch a particular post with the ID
18:27 456 so for obvious reasons this is
18:31 called a nested route right because we
18:33 Nest different types of information in
18:36 different levels to express different
18:39 semantic meanings and it results in
18:42 different kinds of responses all right
18:44 that's all there is to it in nested
18:46 routing it's pretty convenient and you
18:48 will see it getting used pretty much
18:50 everywhere if the API is even has a
18:53 medium level of complexity next up we
18:55 have an interesting concept called route
18:57 versioning and deprecation so let's just
18:60 fire these two apis and we'll jump into
19:02 the concept of
19:03 that okay so if you look into
19:07 this so this looks pretty much like our
19:11 earlier request except it has this
19:14 keyword which is V1 it says API slv1
19:18 SL products and if you look at the
19:22 second request it says V2 right so this
19:26 is called route versioning it is a very

19:29 common practice in API endpoints uh
19:33 servers which has a rest API uh
19:36 interface and the the use of this is if
19:40 we look at these two requests and
19:42 responses for that we'll understand why
19:44 do we use versioning in servers right in
19:48 API endpoints if you look at the
19:50 response of this we are saying / API
19:53 slv1 products and this is what the
19:56 response looks like it has field data
19:58 and and it has an array and each array
20:02 has an object Json with ID name price ID
20:06 name price right and in the second
20:09 request which is the V2 request we have
20:13 again data and we have ID title price
20:16 right in the first one we had ID name
20:19 price in the second one we have ID title
20:21 price and now it is a pretty trivial
20:23 example usually the use of versioning is
20:26 let's say you have an API endpoint and
20:28 you are returning data in a particular
20:30 format right and later on new
20:32 requirements came in let's say you were
20:34 earlier serving a web app and future in
20:38 the future you are serving a react
20:39 native app right an Android app or a
20:41 flutter app for that entity or that
20:43 device you had to change the response of
20:45 your data so your first option is
20:47 changing the whole route right changing
20:49 the whole route matching part so let's
20:51 say you can say/ API SL new products
20:54 right or what you can do is you can add
20:58 that to your versioning you can say in
21:00 the version one of our API we were
21:02 serving responses in this format and new
21:05 requirements came in and in version two
21:07 we are serving in this format one thing
21:09 it expresses your intention very clearly
21:12 you have the version one of response

21:14 your version one of response and you
21:15 have version 2f response the second part
21:18 is you did not have to change your whole
21:21 route you did not have to do/ API SL new
21:24 products using this versioning concept
21:28 you can even eventually deprecate V1 you
21:31 can let's say uh send a notice to your
21:34 front end Engineers that after the V2 is
21:36 released V1 is deprecated so in the next
21:40 release all the engineers can eventually
21:42 migrate to V2 so they have this window a
21:45 particular window where they have the
21:48 opportunity to migrate the request
21:51 endpoints to the V2 format and
21:53 eventually you will completely get rid
21:55 of V1 and you'll make the V2 to V1 right
21:58 so you have a very stable and complete
22:01 workflow to add new structure to your
22:04 API endpoints add breaking changes to
22:06 your API endpoints using this workflow
22:09 and your engineers your client have
22:12 window where they have the opportunity
22:14 to migrate into the new structure that's
22:17 the use of route versioning and
22:19 deprecation in the end we have something
22:21 called catch all route so if we hit
22:26 this and if we look at the request and
22:29 in
22:29 response so we what we are doing here is
22:33 we are sending requests to a route which
22:37 the server does not serve for right we
22:39 are doing SL a/ V3 SL products at this
22:43 point the server does not serve
22:45 responses for this route right it does
22:47 not have an Handler it does not cater to
22:49 requests coming into this endpoint so
22:51 typically what the server will do is in
22:54 the end after serving all the different
22:56 routes and all the methods uh associated
22:59 with those route in the end it will do

23:00 is slash
23:03 star whatever request after going
23:05 through all the previous route matching
23:08 algorithms whatever request reaches here
23:11 this part it will be mapped to a Handler
23:13 and that Handler will send a
23:16 userfriendly message that this route
23:19 which you are requesting this route does
23:21 not exist right this route are not found
23:23 so instead of just sending a null
23:26 response which is the default Behavior
23:28 if you don't do a catch all handling we
23:31 are just sending a userfriendly message
23:33 to let the client know that we don't
23:35 cater to this endpoint that's all catch
23:37 all route is about and with that we have
23:40 covered pretty much all the concepts
23:42 that one needs to know before diving
23:44 deep into different kinds of routing and
23:47 all the components of routing like query
23:50 parameters path parameters and all the
23:53 nested Dynamic parameters and that's all
23:55 pretty much you need to know to get into
23:57 a backend code base and and understand
23:59 the routing parts and make changes to it
24:02 and add new things